

Job Hunt Tracker - Design Document

A self-hosted job application tracker with email-driven status detection

Matt

August 2026

Overview
Architecture
Project structure
Key data structures
Services
 Background worker
Email classification prompt
Web interface and alerting
 UI layout
 Alerting options
Suggested build order
Estimated running cost

Overview

This document captures the design for a self-built job application tracker, intended to replace manual spreadsheet tracking with:

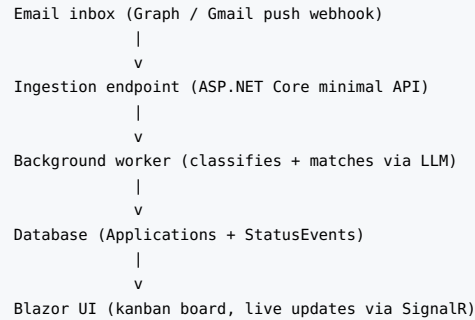
- A kanban-style pipeline view of every application
- Automatic status detection from incoming recruiter/ATS emails (no manual re-typing when a rejection or interview invite lands)
- Structured retention of job descriptions and contact info per application

The stack is C#/ .NET throughout: ASP.NET Core for the API and background processing, Blazor for the UI, EF Core for persistence. This keeps the whole project inside a single, familiar toolchain and avoids introducing a second language just for a personal tool.

Why build instead of buy: existing tools (Teal, Huntr, Simplify) all provide solid manual kanban tracking, but none reliably solve the “read my inbox and update status automatically” problem - that remains a copy-paste chore on every mainstream tracker as of mid-2026. That gap is the actual justification for building a custom tool rather than subscribing to one.

Architecture

The system is a single pipeline: an inbound email triggers a webhook, a background worker classifies and matches it against tracked applications, and the result is written to the database and pushed live to the UI.



Flow, step by step:

1. The email provider (Microsoft Graph or Gmail API) sends a push notification to a webhook endpoint whenever a new message arrives.
2. The endpoint stores the raw email and enqueues its ID onto an in-process channel - it does not block on classification.
3. A `BackgroundService` worker dequeues the email, calls an LLM (Claude Haiku) to classify whether it's job-search related and what status it implies, then attempts to match it to a tracked `Application` by sender domain / company name.
4. If matched with sufficient confidence, a `StatusEvent` row is written and the application's current status is updated. Low-confidence matches are surfaced in a "needs review" queue instead of applied automatically.
5. The Blazor UI, connected via SignalR, receives a push notification and updates the board without a page refresh.

Project structure

```

JobHuntTracker.sln
├── src/
│   ├── JobHuntTracker.Core/           # domain models, interfaces, no dependencies
│   │   ├── Entities/
│   │   │   ├── Application.cs
│   │   │   ├── StatusEvent.cs
│   │   │   ├── Contact.cs
│   │   │   └── EmailMessage.cs
│   │   ├── Enums/
│   │   │   └── ApplicationStatus.cs
│   │   └── Interfaces/
│   │       ├── IApplicationService.cs
│   │       ├── IEmailClassifierService.cs
│   │       ├── IApplicationMatchingService.cs
│   │       └── IJobDescriptionExtractionService.cs
│   ├── JobHuntTracker.Infrastructure/  # EF Core, external API clients
│   │   ├── Data/
│   │   │   ├── AppDbContext.cs
│   │   │   └── Migrations/
│   │   ├── Email/
│   │   │   ├── GraphEmailClient.cs
│   │   │   └── EmailWebhookHandler.cs
│   │   ├── Llm/
│   │   │   ├── ClaudeClient.cs
│   │   │   ├── EmailClassifierService.cs
│   │   │   └── JobDescriptionExtractionService.cs
│   │   └── Services/
│   │       ├── ApplicationMatchingService.cs
│   │       └── ApplicationService.cs
│   ├── JobHuntTracker.Api/            # ASP.NET Core host
│   │   ├── Program.cs
│   │   ├── Endpoints/
│   │   │   ├── ApplicationEndpoints.cs
│   │   │   └── EmailWebhookEndpoints.cs
│   │   ├── Workers/
│   │   │   └── EmailProcessingWorker.cs # BackgroundService, drains the queue
│   │   └── appsettings.json
│   └── JobHuntTracker.Web/            # Blazor Server frontend
│       ├── Components/
│       │   ├── KanbanBoard.razor
│       │   ├── ApplicationCard.razor
│       │   ├── NeedsReviewPanel.razor
│       │   └── ApplicationDetail.razor
│       ├── Hubs/
│       │   └── JobTrackerHub.cs       # SignalR hub
│       └── Program.cs
└── tests/
    └── JobHuntTracker.Tests/

```

Four projects is deliberately the ceiling here. `Core` has zero external package references, which keeps domain logic testable and keeps the design provider-agnostic - swapping Microsoft Graph for the Gmail API later touches `Infrastructure` only, not `Core` or `Api`.

Key data structures

```

public enum ApplicationStatus
{
    Saved, Applied, Screening, InterviewScheduled,
    Interviewed, OfferReceived, Rejected, Ghosted, Withdrawn
}

public class Application
{
    public Guid Id { get; set; }
    public string CompanyName { get; set; } = "";
    public string RoleTitle { get; set; } = "";
    public string SourceUrl { get; set; } = "";
    public string JobDescriptionRaw { get; set; } = "";
    public JobDescriptionExtract? JdExtract { get; set; } // stored as json
    public ApplicationStatus CurrentStatus { get; set; }
    public DateTime DateApplied { get; set; }
    public string? ResumeVersionUsed { get; set; }
    public string? Notes { get; set; }
    public List<Contact> Contacts { get; set; } = [];
    public List<StatusEvent> StatusHistory { get; set; } = [];
    public DateTime CreatedAt { get; set; }
    public DateTime UpdatedAt { get; set; }
}

// what the LLM extracts once, at creation time
public record JobDescriptionExtract(
    string[] RequiredSkills,
    string[] NiceToHave,
    string? SalaryRangeText,
    string? Location,
    string? RemotePolicy,
    int? YearsExperience
);

// one row per detected status change -- this is the audit trail
public class StatusEvent
{
    public Guid Id { get; set; }
    public Guid ApplicationId { get; set; }
    public ApplicationStatus Status { get; set; }
    public DateTime DetectedAt { get; set; }
    public Guid? SourceEmailId { get; set; } // null if manually set
    public double? Confidence { get; set; } // LLM confidence, null if manual
    public string? RawSnippet { get; set; } // the sentence that triggered this
}

public class EmailMessage
{
    public Guid Id { get; set; }
    public string ExternalMessageId { get; set; } = ""; // Graph/Gmail id, for
    idempotency
    public string FromAddress { get; set; } = "";
    public string Subject { get; set; } = "";
    public DateTime ReceivedAt { get; set; }
    public string BodyText { get; set; } = "";
    public DateTime? ProcessedAt { get; set; }
    public Guid? MatchedApplicationId { get; set; }
    public string? ClassificationResultJson { get; set; } // raw LLM output, for debugging
}

public class Contact
{
    public Guid Id { get; set; }
    public Guid ApplicationId { get; set; }
    public string Name { get; set; } = "";
    public string? Email { get; set; }
    public string? Role { get; set; } // recruiter, hiring manager, etc.
}

```

Design note: `StatusEvent` is kept as an append-only table rather than just overwriting `Application.CurrentStatus` directly. This gives a free timeline view, lets the UI show confidence and provenance per transition, and means a bad LLM classification never silently overwrites the only copy of the truth - it's just one row that can be reviewed or discarded.

Services

```

public interface IEmailClassifierService
{
    Task<EmailClassification> ClassifyAsync(EmailMessage email, CancellationToken ct);
}

public record EmailClassification(
    bool IsJobRelated,
    string? InferredCompanyName,
    ApplicationStatus? InferredStatus,
    double Confidence,
    string? Reasoning
);

public interface IApplicationMatchingService
{
    // fuzzy match against sender domain, company name, subject line
    Task<Application?> FindMatchAsync(
        EmailMessage email, EmailClassification classification, CancellationToken ct);
}

public interface IJobDescriptionExtractionService
{
    Task<JobDescriptionExtract> ExtractAsync(string rawJobDescription, CancellationToken
ct);
}

public interface IApplicationService
{
    Task<Application> CreateAsync(CreateApplicationRequest request, CancellationToken ct);

    Task RecordStatusEventAsync(
        Guid applicationId, ApplicationStatus status, Guid? sourceEmailId,
        double? confidence, string? snippet, CancellationToken ct);

    Task<List<Application>> GetBoardAsync(CancellationToken ct);
}

```

Background worker

The worker that ties everything together is deliberately simple: a `Channel<Guid>` queue that the webhook endpoint writes to, and a loop that reads, classifies, matches, and records.

```

protected override async Task ExecuteAsync(CancellationToken ct)
{
    await foreach (var emailId in _queue.Reader.ReadAllAsync(ct))
    {
        var email = await _db.EmailMessages.FindAsync([emailId], ct);
        var classification = await _classifier.ClassifyAsync(email!, ct);

        if (!classification.IsJobRelated)
        {
            email!.ProcessedAt = DateTime.UtcNow;
            continue;
        }

        var match = await _matcher.FindMatchAsync(email!, classification, ct);
        if (match is not null && classification.InferredStatus is { } status)
        {
            await _applications.RecordStatusEventAsync(
                match.Id, status, email!.Id, classification.Confidence,
                classification.Reasoning, ct);
        }

        email!.ProcessedAt = DateTime.UtcNow;
        email.MatchedApplicationId = match?.Id;
        await _db.SaveChangesAsync(ct);
    }
}

```

Using an in-process `Channel<Guid>` instead of a durable message broker (e.g. Hangfire) is a deliberate simplification for a solo, personal-scale tool - a missed email on restart is not a meaningful risk here. If that changes, only `Program.cs`'s registration and the queue implementation need to change; the worker's logic does not.

Email classification prompt

A structured-output call to Claude Haiku, run once per incoming email:

```
System: You classify emails related to a job search. Respond with ONLY
valid JSON matching this schema:
{
  "isJobRelated": bool,
  "companyName": string|null,
  "status": "Applied"|"Screening"|"InterviewScheduled"|"Interviewed"|"
    OfferReceived"|"Rejected"|"Ghosted"|null,
  "confidence": number 0-1,
  "reasoning": string
}

User: From: {email.From}
Subject: {email.Subject}
Body: {email.BodyText[..2000]}
```

The email body is truncated to ~2000 characters - the classification signal is almost always in the first few sentences, and truncating keeps per-call token cost negligible even at high email volume.

Confidence threshold: a cutoff (suggested starting point: 0.75) decides routing:

- **Above threshold:** status change is applied automatically; a toast notification confirms it in the UI.
- **Below threshold:** the detection is written to a “needs review” queue instead of applied, requiring a one-click confirm or dismiss from the user. This is the main tuning knob for how much the system is trusted to act unattended, and can be adjusted over time as classification accuracy is observed.

Web interface and alerting

`JobHuntTracker.Web` is a Blazor Server application - a real web UI, run locally (or on a small VPS) and accessed in a browser. Live updates are delivered via a SignalR hub (`JobTrackerHub`): when the worker records a `StatusEvent` , connected clients are notified and the kanban board updates without a manual refresh.

UI layout

- A notification indicator (bell icon with count) in the header
- A “needs review” panel above the board surfacing low-confidence detections, each with inline **Confirm** / **Dismiss** actions
- A kanban board with one column per `ApplicationStatus` , each card showing company, role, and last-updated date

Alerting options

Two levels of alerting are possible, and the recommendation is to build them in this order:

1. **Browser-based (build first):** toast notifications and the needs-review panel inside the Blazor page itself. This requires no additional platform work and validates that the classification pipeline is accurate enough to be useful before investing further.
2. **OS-level desktop notifications (build later, optional):** wrap the same Razor components in a **.NET MAUI Blazor Hybrid** shell, giving a system tray icon and native OS toast notifications that work even when no browser tab is open. This reuses the majority of the existing Blazor components - the SignalR client subscribes the same way regardless of host - so it is an additive step rather than a

rebuild.

Suggested build order

1. **Core data model + manual CRUD.** `Application`, `StatusEvent`, `Contact` entities, EF Core migrations, and a Blazor kanban board with manual status changes only. This alone already beats a spreadsheet.
2. **Job description extraction.** One LLM call at application-creation time to populate `JobDescriptionExtract`. Validates the LLM integration pattern in isolation before adding the more complex email pipeline.
3. **Email ingestion + classification pipeline.** Webhook endpoint, background worker, classifier prompt, matching logic, and the needs-review queue. This is the highest-value and highest-uncertainty piece - expect to tune the confidence threshold and prompt after seeing real emails run through it.
4. **Live UI updates.** SignalR hub wiring so the board and needs-review panel update without a refresh.
5. **Desktop wrapper (optional).** MAUI Blazor Hybrid shell with tray icon and native notifications, once the pipeline above is proven reliable.

Estimated running cost

- **Email API access:** free (Microsoft Graph / Gmail API personal use tiers)
- **LLM classification:** roughly 5-20 emails/day at Claude Haiku pricing - a few cents per month in realistic usage
- **Hosting:** free if run locally / on an always-on home machine; a few dollars/month on a small VPS if remote access is wanted

Total estimated recurring cost: **under \$10/month**, and likely closer to \$0 if run locally.